

Task01

Create RDD from the list

```
lst = [1,2,3,4,5]  
print(lst)  
type(lst)
```

start sparkContext (entrypoint to create RDD)

```
sparkcon = spark.sparkContext  
type = (sparkcon)
```

assign RDD

```
rdd1 = sparkcon.parallelize(lst)  
type = (rdd1)
```

will collect the list and assign

```
rdd1.collect()  
rdd2 = sc.parallelize(lst)  
type = (rdd2)  
rdd2.collect()
```

Number of partitions in RDD

```
rdd2.getNumPartitions()
```

How RDD are distributed

```
rdd2.glom().collect()
```

create fixed number of partitions (control partitions)

```
rdd2 = sc.parallelize(lst, 2)
```

task02

Create RDD from text file

Go to main Icon and advanced – enable dbfs

Catalog - upload text file and copy the location

```
rdd3 = sc.textFile('dbfs:/FileStore/filename.txt')  
rdd3.collect()
```

Number of partitions in RDD

```
Rdd3.getNumPartitions()
```

create fixed number of partitions (control partitions)

```
rdd3.getNumPartitions()  
rdd3 = sc.textFile('dbfs:/FileStore/filename.txt', 2)
```

How RDD are distributed

```
rdd2.glom().collect()
```

Transformations in PySpark

What is a Transformation?

A **transformation** is an operation that:

- Takes an RDD/DataFrame as input
- Returns a **new RDD/DataFrame**
- Is **lazy** (not executed immediately)

Spark only executes transformations when an **action** is called.

Key Characteristics

Feature	Meaning
Lazy Evaluation	Not executed until an action runs
Immutable	Original data is not changed
Returns new object	Creates a new RDD/DataFrame

Transformation Flow

Data → Transformation → Transformation → Action → Result

Common Transformations on RDD

1. map()

Transforms each element

```
rdd = sc.parallelize([1,2,3,4])  
rdd2 = rdd.map(lambda x: x * 2)
```

Output: [2, 4, 6, 8]

2. filter()

Keeps only matching elements

```
rdd3 = rdd.filter(lambda x: x % 2 == 0)
```

Output: [2, 4]

3. flatMap()

Splits and flattens data

```
rdd4 = rdd.flatMap(lambda x: (x, x*10))
```

Output: [1,10,2,20,3,30...]

4. union()

Combines two RDDs

```
rddA.union(rddB)
```

5. distinct()

Removes duplicates

```
rdd.distinct()
```

Common Transformations on DataFrames

1. select()

```
df.select("name", "salary")
```

2. withColumnn()

```
from pyspark.sql.functions import col
df2 = df.withColumnn("bonus", col("salary") * 0.1)
```

3. filter() / where()

```
df.filter(col("salary") > 50000)
```

4. groupBy()

```
df.groupBy("department").sum("salary")
```

5. join()

```
df1.join(df2, "emp_id", "inner")
```

RDD vs DataFrame Transformations

RDD	DataFrame
map	withColumnn
filter	filter
flatMap	explode
reduceByKey	groupBy
union	union

Difference: Transformation vs Action

Transformation	Action
map()	collect()
filter()	show()
select()	count()
groupBy()	write()

Example Demo

```
rdd = sc.parallelize([1,2,3,4,5])

transformed_rdd = rdd.map(lambda x: x * 2).filter(lambda x: x > 5)

# No execution yet

print(transformed_rdd.collect()) # Action triggers execution
```

Summary

Transformations define HOW data will be processed
They are lazy and only run when an action is called
They build execution plans

Flatmap, map , reducedbykey

```
# Read the text file
rdd = sc.textFile("dbfs:/FileStore/flatfile.txt") # change path if needed

# 1. flatMap - split lines into words
words = rdd.flatMap(lambda line: line.split(" "))

# 2. map - convert to (word, 1)
word_pairs = words.map(lambda word: (word, 1))

# 3. reduceByKey - count words
word_count = word_pairs.reduceByKey(lambda a, b: a + b)

# Show result
print(word_count.collect())
```

Step-by-step Processing

Input file:

Hello Spark
Hello PySpark
Spark is fast
PySpark is powerful
Hello Spark PySpark

After flatMap:

```
['Hello','Spark','Hello','PySpark','Spark','is','fast',  
'PySpark','is','powerful','Hello','Spark','PySpark']
```

After map:

```
('Hello',1), ('Spark',1), ('Hello',1), ('PySpark',1) ...
```

After reduceByKey (Final Output):

```
[('Hello', 3),  
( 'Spark', 3),  
( 'PySpark', 3),  
( 'is', 2),  
( 'fast', 1),  
( 'powerful', 1)]
```

Visual Flow

TXT File



flatMap(split)



map(word,1)



reduceByKey(sum)



Final Word Count

What is a Lineage Graph in Spark?

A **Lineage Graph** is Spark's internal record that shows:

How an RDD was created from its source through a series of transformations.

It is the **transformation history of data**.

Simple Definition

Lineage Graph = the complete chain of operations that created an RDD

Spark uses it to **rebuild data if something fails**.

Why Lineage Graph exists

Spark does NOT always store intermediate results.

If a partition is lost (node failure), Spark:

1. Looks at the lineage graph
2. Re-runs the same transformations
3. Recreates the lost data

This provides **fault tolerance**.

Example

```
rdd = sc.textFile("dbfs:/FileStore/flatfile.txt")
words = rdd.flatMap(lambda x: x.split(" "))
pairs = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

Lineage Graph Flow

textFile



flatMap



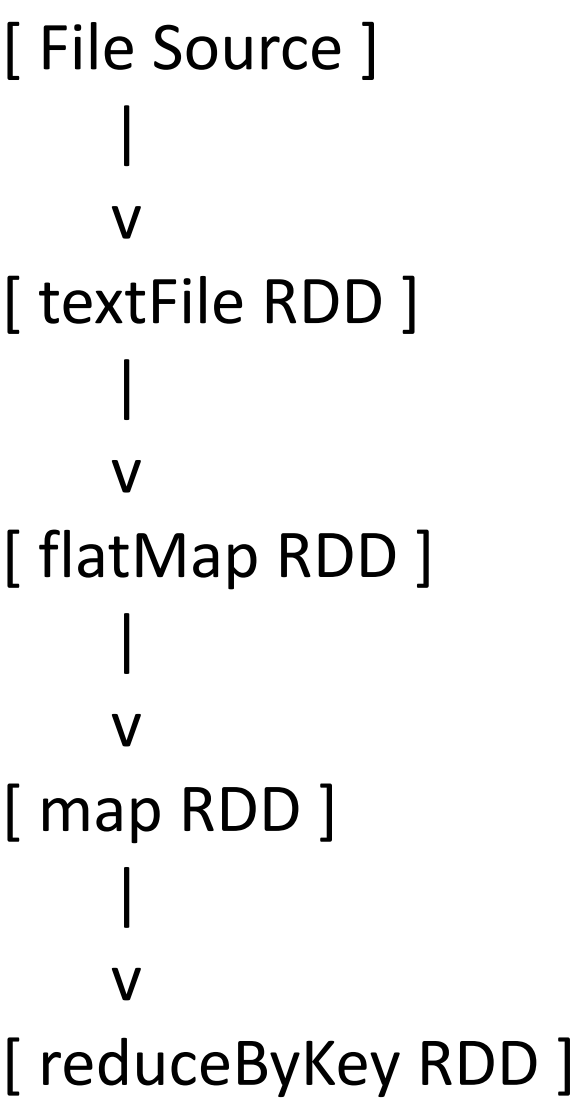
map



reduceByKey

Each step depends on the previous one.

Visual Representation



This chain is the **lineage graph**.

How to View Lineage

Use in PySpark:

```
print(counts.toDebugString())
```

Sample output:

```
b'(2) PythonRDD[63] at RDD at PythonRDD.scala:63 []\n | MapPartitionsRDD[62] at mapPartitions at\nPythonRDD.scala:252 []\n | ShuffledRDD[61] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +- (2)\nPairwiseRDD[60] at wrapper at /root/.ipykernel/1717/command-4915190394772677-2943871905:4 []\n | \nPythonRDD[59] at wrapper at /root/.ipykernel/1717/command-4915190394772677-2943871905:4 []\n | \ndbfs:/FileStore/flatfile.txt MapPartitionsRDD[58] at textFile at NativeMethodAccessorImpl.java:0 []\n | \ndbfs:/FileStore/flatfile.txt HadoopRDD[57] at textFile at NativeMethodAccessorImpl.java:0 []'
```

Lineage graph

Feature	Lineage Graph
Purpose	Tracks how data was created
Used for	Fault tolerance & recovery
Shows	Parent-child RDD relationship

Lineage graph is the sequence of transformations Spark records for each RDD, enabling it to recompute lost data instead of storing it, thus providing fault tolerance.

What is DAG in Apache Spark?

DAG = Directed Acyclic Graph

In Spark, a **DAG** is the **execution plan of your job** that shows:

The order in which Spark will run transformations and actions, and how they depend on each other.

Simple Definition

DAG is a graph that represents all Spark operations and their dependencies in a one-way flow with no loops.

- Directed → Data flows in one direction
- Acyclic → No operation goes back to itself
- Graph → Nodes + connections

Why Spark uses DAG

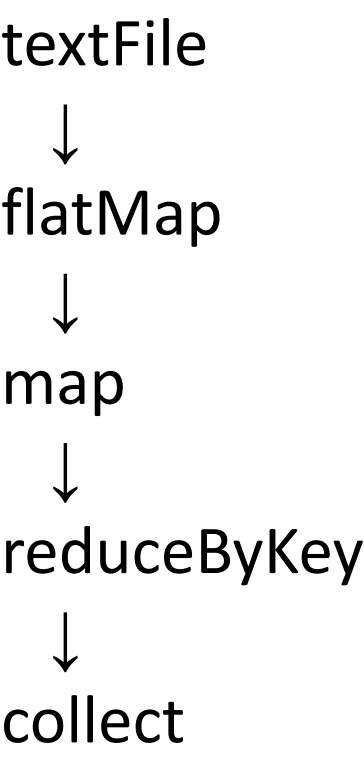
Spark uses DAG to:

- Optimize job execution
- Split a job into stages
- Decide parallel execution
- Minimize shuffles

Example

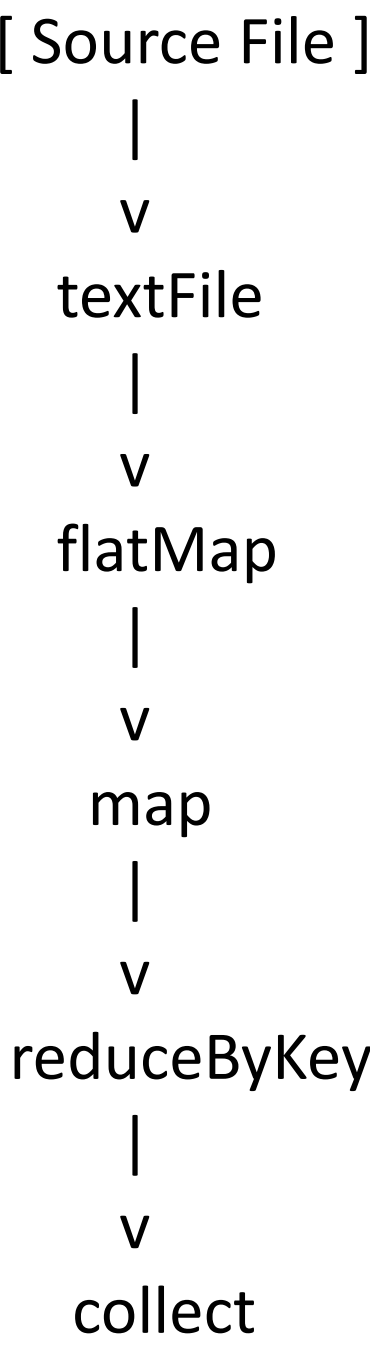
```
rdd = sc.textFile("dbfs:/FileStore/flatfile.txt")
words = rdd.flatMap(lambda x: x.split(" "))
pairs = words.map(lambda w: (w, 1))
result = pairs.reduceByKey(lambda a, b: a + b)
result.collect()
```

DAG Flow



This flow is the **DAG**.

Visual DAG



DAG vs Lineage (Quick Difference)

Feature	DAG	Lineage Graph
Focus	Execution process	Transformation history
Used for	Scheduling & optimization	Fault tolerance
Created when	Action is called	RDD is created

How Spark processes DAG

1. Action is triggered (collect(), count())
 2. Spark builds DAG
 3. DAG Scheduler splits it into **stages**
 4. Each stage is divided into **tasks**
 5. Tasks execute in parallel
-

One-liner

DAG in Spark is a logical execution plan that represents the sequence and dependency of transformations in a job without any cycles.

Cooking process:

Cut vegetables → Fry → Add spices → Serve

No step returns backward → This is a DAG.

Short Summary

- DAG = Spark job execution blueprint
- Ensures efficient processing
- Controls task scheduling

Introduction to Databricks and PySpark

```
# First PySpark program
from pyspark.sql import SparkSession

# Create SparkSession
spark = SparkSession.builder \
    .appName("MyFirstApp") \
    .getOrCreate()

print("Spark version:", spark.version)
spark.stop()
```

RDD Fundamentals

```
# RDD creation and basic operations
from pyspark import SparkContext

sc = SparkContext.getOrCreate()

# Create RDD
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data)
print("RDD partitions:", rdd.getNumPartitions())
print("RDD content:", rdd.collect())
```

Create RDD from List

```
# Different ways to create RDD from collections
sc = SparkContext.getOrCreate()

# Method 1: parallelize
list_data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd1 = sc.parallelize(list_data)

# Method 2: with partitions
rdd2 = sc.parallelize(list_data, 4) # 4 partitions

print("Number of Partitions:", rdd2.getNumPartitions())
print("Data distribution:", rdd2.glom().collect()) # Show data per
partition
```

Control Partitions In RDD

```
# Partition control examples
rdd = sc.parallelize(range(100), 10) # 10 partitions initially

# Repartition to 5 partitions
rdd_repartitioned = rdd.repartition(5)
print("After repartition:", rdd_repartitioned.getNumPartitions())
```

```
# Coalesce to 3 partitions (more efficient than repartition)
rdd_coalesced = rdd.coalesce(3)
print("After coalesce:", rdd_coalesced.getNumPartitions())
```

Create RDD from Textfile

```
# Create RDD from text file
# Create sample text file first
with open("sample.txt", "w") as f:
    f.write("Hello Spark\n")
    f.write("Learning PySpark\n")
    f.write("RDD operations\n")

# Read text file
text_rdd =
sc.textFile("/Volumes/test_catalog/test_schema/test_volume/sample_data_1.csv")

# Count lines
print("Line count:", text_rdd.count())

# Show content
print("File content:")
for line in text_rdd.collect():
    print(line)
```

Transformations On RDD

```
# Common RDD transformations
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd = sc.parallelize(data)

# Map transformation
squared_rdd = rdd.map(lambda x: x * x)
print("Squared:", squared_rdd.collect())

# Filter transformation
even_rdd = rdd.filter(lambda x: x % 2 == 0)
print("Even numbers:", even_rdd.collect())
```

```
# FlatMap transformation
words_rdd = sc.parallelize(["hello world", "spark tutorial"])
words_split = words_rdd.flatMap(lambda x: x.split(" "))
print("Words:", words_split.collect())
```

Lineage Graph

```
# Understanding RDD lineage
rdd1 = sc.parallelize([1, 2, 3, 4, 5])
rdd2 = rdd1.map(lambda x: x * 2)
rdd3 = rdd2.filter(lambda x: x > 5)
rdd4 = rdd3.map(lambda x: x + 10)
```

```
# Check lineage
print("Lineage:")
print(rdd4.toDebugString())
```

```
# This shows the transformation history
```

Understanding DAG Fundamentals

```
# DAG (Directed Acyclic Graph) example
rdd = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Build complex transformation pipeline
result = (rdd
    .map(lambda x: (x, 1))           # Stage 1
    .filter(lambda x: x[0] > 3)      # Stage 1
    .reduceByKey(lambda a, b: a + b) # Stage 2 - causes shuffle
    .map(lambda x: (x[1], x[0]))     # Stage 2
)

print("Result:", result.collect())
print("DAG visualization available in Spark UI")
```

MapReduce Working

```
# MapReduce pattern in Spark
```

```
# Word count example (classic MapReduce)
text_data = ["hello world", "hello spark", "spark tutorial", "world peace"]
text_rdd = sc.parallelize(text_data)

# MapReduce steps
word_counts = (text_rdd
               .flatMap(lambda line: line.split(" ")) # Map phase
               .map(lambda word: (word, 1))           # Map phase
               .reduceByKey(lambda a, b: a + b)       # Reduce phase
               )

print("Word counts:", word_counts.collect())
```

ReduceByKey Vs GroupByKey

```
# Performance comparison
data = [("a", 1), ("b", 2), ("a", 3), ("b", 4), ("c", 5)]
rdd = sc.parallelize(data)

# reduceByKey - more efficient (combines data before shuffle)
sum_reduce = rdd.reduceByKey(lambda a, b: a + b)
print("reduceByKey result:", sum_reduce.collect())

# groupByKey - less efficient (shuffles all data)
sum_group = rdd.groupByKey().mapValues(lambda x: sum(x))
print("groupByKey result:", sum_group.collect())
```

GroupByKey

```
# GroupByKey detailed example
student_marks = [
    ("math", 85), ("science", 90), ("math", 78),
    ("english", 88), ("science", 92), ("english", 76)
]
rdd = sc.parallelize(student_marks)

# Group marks by subject
grouped = rdd.groupByKey()

# Calculate average per subject
average_marks = grouped.mapValues(lambda marks: sum(marks) / len(marks))
print("Average marks by subject:", average_marks.collect())
```

Filter Transformation On RDD

```
# Various filter operations
```

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
rdd = sc.parallelize(data)
```

```
# Multiple filter conditions
```

```
filtered = rdd.filter(lambda x: x > 3 and x < 8)
```

```
print("Numbers between 3 and 8:", filtered.collect())
```

```
# Filter with complex condition
```

```
numbers = sc.parallelize([15, 23, 18, 42, 7, 56, 31])
```

```
even_greater_than_20 = numbers.filter(lambda x: x % 2 == 0 and x > 20)
```

```
print("Even numbers > 20:", even_greater_than_20.collect())
```

SortBy & SortByKey

```
# Sorting operations
```

```
# SortBy
```

```
data = [("John", 25), ("Alice", 30), ("Bob", 20), ("Diana", 35)]
```

```
rdd = sc.parallelize(data)
```

```
# Sort by age
```

```
sorted_by_age = rdd.sortBy(lambda x: x[1])
```

```
print("Sorted by age:", sorted_by_age.collect())
```

```
# SortByKey (for key-value pairs)
```

```
kv_rdd = sc.parallelize([(3, "c"), (1, "a"), (2, "b")])
```

```
sorted_by_key = kv_rdd.sortByKey()
```

```
print("Sorted by key:", sorted_by_key.collect())
```

Extract Top Bottom From RDD

```
# Top and bottom elements
```

```
data = [45, 23, 78, 12, 89, 5, 67, 34, 90, 21]
```

```
rdd = sc.parallelize(data)
```

```
# Top 3 elements
```

```
top_3 = rdd.top(3)
```

```
print("Top 3:", top_3)
```

```
# Bottom 3 elements
```

```
bottom_3 = rdd.takeOrdered(3)
print("Bottom 3:", bottom_3)

# Top 3 with custom ordering
students = [("John", 85), ("Alice", 92), ("Bob", 78), ("Diana", 95)]
students_rdd = sc.parallelize(students)
top_students = students_rdd.takeOrdered(2, key=lambda x: -x[1])
print("Top 2 students:", top_students)
```

Save RDD as Textfile

```
# Saving RDD to file system
data = ["Apache Spark", "PySpark", "RDD", "DataFrame", "MLlib"]
rdd = sc.parallelize(data)

# Save as text file
output_path = "output/rdd_output"
rdd.saveAsTextFile(output_path)

# Verify by reading back
saved_rdd = sc.textFile(output_path)
print("Saved content:", saved_rdd.collect())
```

Coalesce And Repartition

```
# Partition management

# Create RDD with many partitions
rdd = sc.parallelize(range(1000), 10) # 10 partitions
print("Initial partitions:", rdd.getNumPartitions())

# Coalesce - reduces partitions efficiently (no shuffle)
coalesced = rdd.coalesce(2)
print("After coalesce to 2:", coalesced.getNumPartitions())
```

```
# Repartition - can increase or decrease partitions (with shuffle)
repartitioned = rdd.repartition(5)
print("After repartition to 5:", repartitioned.getNumPartitions())

# Repartition by column (for Pair RDDs)
data = [("A", 1), ("B", 2), ("A", 3), ("C", 4), ("B", 5)]
pair_rdd = sc.parallelize(data)
repartitioned_by_key = pair_rdd.partitionBy(3)
print("Partitioned by key, partitions:",
repartitioned_by_key.getNumPartitions())
```

Accumulators

```
# Accumulators for distributed counters

# Initialize accumulator
error_counter = sc.accumulator(0)
empty_line_counter = sc.accumulator(0)

def process_line(line):
    global error_counter, empty_line_counter

    if not line.strip():
        empty_line_counter.add(1)
        return None
    try:
        # Some processing that might fail
        return line.upper()
    except:
        error_counter.add(1)
        return None

# Process data
data = ["hello", "", "world", "spark", ""]
rdd = sc.parallelize(data)
processed = rdd.map(process_line).filter(lambda x: x is not None)
```

```
print("Processed data:", processed.collect())
print("Empty lines:", empty_line_counter.value)
print("Errors:", error_counter.value)
```

Broadcast Variables

```
# Broadcast variables for efficient data distribution
# Large lookup data that should be available on all nodes
country_codes = {
    "US": "United States",
    "IN": "India",
    "UK": "United Kingdom",
    "CA": "Canada"
}

# Broadcast the lookup table
broadcast_countries = sc.broadcast(country_codes)

# Sample data with country codes
data = [("John", "US"), ("Alice", "UK"), ("Bob", "IN"), ("Charlie", "CA")]
rdd = sc.parallelize(data)

# Use broadcast variable in transformation
def map_country(name_code):
    name, code = name_code
    country_name = broadcast_countries.value.get(code, "Unknown")
    return (name, country_name)

result = rdd.map(map_country)
print("Mapped countries:", result.collect())
```

What is a DataFrame in PySpark

A **DataFrame** in PySpark is a **distributed collection of data organized into named columns**, similar to a table in SQL or a dataframe in Pandas — but **distributed across a cluster**.

It is built on top of **RDDs** and optimized using:

- **Catalyst Optimizer** (for query optimization)
- **Tungsten** (for memory + execution optimization)

Key Features of PySpark DataFrame

1. Schema-based (rows + columns)

```
+-----+-----+
| Name | Age |
+-----+-----+
| Raj  | 30  |
+-----+-----+
```

2. Distributed

Data is not stored on one machine — Spark splits it into partitions across the cluster.

3. Lazy evaluation

Transformations do not run immediately; they run only when an **action** is executed.

4. Optimized execution

Spark uses:

- Catalyst (query optimization)
- Tungsten (CPU + memory optimizations)

5. Immutable

Once created, DataFrames cannot be changed; they create new DataFrames.

Creating a DataFrame

From Python list

```
data = [("Alice", 25), ("Bob", 30)]
columns = ["Name", "Age"]
df = spark.createDataFrame(data, columns)
df.show()
```

From CSV

```
df = spark.read.csv("data.csv", header=True, inferSchema=True)
```

From JSON

```
df = spark.read.json("data.json")
```

Common DataFrame Operations

Select columns

```
df.select("Name", "Age").show()
```

Filter rows

```
df.filter(df.Age > 25).show()
```

Add new column

```
df.withColumn("AgePlus10", df.Age + 10).show()
```

Group by

```
df.groupBy("Department").count().show()
```

Actions vs Transformations

Transformations (lazy)

- select
- filter
- withColumn
- join

Actions (trigger execution)

- show
- count
- collect
- write

Why is DataFrame better than RDD

Feature	RDD	DataFrame
Schema	 No	 Yes

Optimization	✗ No	✓ Catalyst + Tungsten
Performance	✗ Slower	✓ Faster
Ease of use	✗ Requires code	✓ SQL-like API

DataFrame Introduction & CSV Reading

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

spark = SparkSession.builder.appName("DataFrameDemo").getOrCreate()

# Read CSV with inference
df = spark.read.csv("data/sample.csv", header=True, inferSchema=True)
print("Schema:")
df.printSchema()
print("Data insights:")
df.show(5)
print("Summary stats:")
df.describe().show()
```

Select Columns From DataFrame

```
# Column selection examples
from pyspark.sql.functions import col

# Create sample DataFrame
data = [("John", 25, "Engineer", 50000),
        ("Alice", 30, "Manager", 75000),
        ("Bob", 35, "Analyst", 60000)]
df = spark.createDataFrame(data, ["name", "age", "job", "salary"])

# Different selection methods
df.select("name", "age").show()
df.select(col("name"), col("salary")).show()
df.select(df.name, df.age + 1).show() # With expression
```

Add Modify Columns In DataFrame

```
from pyspark.sql.functions import when, lit, concat

# Add new columns
df_with_bonus = df.withColumn("bonus", df.salary * 0.1)
df_with_fullname = df.withColumn("full_name", concat(df.name, lit(" - Employee")))

# Modify existing columns
df_modified = df.withColumn("salary", df.salary + 1000)
df_with_age_group = df.withColumn("age_group",
    when(df.age < 30, "Young")
    .when(df.age < 40, "Middle")
    .otherwise("Senior"))

df_with_age_group.show()
```

Drop Columns In DataFrame

```
# Drop single column
df_dropped = df.drop("job")

# Drop multiple columns
df_dropped_multiple = df.drop("job", "age")

# Drop with condition
columns_to_drop = [col for col in df.columns if col.startswith('temp_')]
df_cleaned = df.drop(*columns_to_drop)

df_dropped.show()
```

Rename Columns In DataFrame

```
# Rename columns
df_renamed = df.withColumnRenamed("name", "employee_name") \
    .withColumnRenamed("salary", "annual_salary")

# Rename multiple columns
new_columns = {"name": "full_name", "age": "employee_age"}
```

```
for old_col, new_col in new_columns.items():
    df = df.withColumnRenamed(old_col, new_col)

df_renamed.show()
```

Sort Columns In DataFrame

```
from pyspark.sql.functions import desc, asc

# Sort by single column
df_sorted = df.orderBy("salary")
df_sorted_desc = df.orderBy(desc("salary"))

# Sort by multiple columns
df_multi_sorted = df.orderBy("job", desc("salary"))

# Using sort() method
df_sorted_alt = df.sort("age")

df_multi_sorted.show()
```

Remove Duplicates From DataFrame

```
# Remove duplicate rows
data_with_dups = [
    ("John", 25, "Engineer"), ("Alice", 30, "Manager"),
    ("John", 25, "Engineer"), ("Bob", 35, "Analyst")
]
df_dups = spark.createDataFrame(data_with_dups, ["name", "age", "job"])

# Remove all duplicates
```

```
df_no_dups = df_dups.distinct()

# Remove duplicates based on specific columns
df_deduped_cols = df_dups.dropDuplicates(["name"])

df_no_dups.show()
```

Combine DataFrames

```
# Union operations
data1 = [("John", 25), ("Alice", 30)]
data2 = [("Bob", 35), ("Diana", 28)]

df1 = spark.createDataFrame(data1, ["name", "age"])
df2 = spark.createDataFrame(data2, ["name", "age"])

# Union (removes duplicates)
df_union = df1.union(df2)

# UnionAll (keeps duplicates)
df_union_all = df1.unionAll(df2)

df_union.show()
```

Pattern Based Filters

```
from pyspark.sql.functions import col

# Regex pattern filters
df_pattern = df.filter(col("name").rlike("^A.*")) # Names starting with A
df_like = df.filter(col("job").like("%ana%")) # Jobs containing 'ana'
df_startswith = df.filter(col("name").startswith("A"))
df_endswith = df.filter(col("name").endswith("e"))

df_pattern.show()
```

Window Functions

```
from pyspark.sql.window import Window
```

```
from pyspark.sql.functions import row_number, rank, dense_rank, lag, lead

# Define window specification
window_spec = Window.partitionBy("job").orderBy("salary")

df_window = df.withColumn("row_num", row_number().over(window_spec)) \
               .withColumn("rank", rank().over(window_spec)) \
               .withColumn("dense_rank", dense_rank().over(window_spec)) \
               .withColumn("prev_salary", lag("salary").over(window_spec))

df_window.show()
```

Fill Null Values

```
from pyspark.sql.functions import coalesce

# Create DataFrame with nulls
data_with_nulls = [("John", 25, None), ("Alice", None, 75000), (None, 35,
60000)]
df_nulls = spark.createDataFrame(data_with_nulls, ["name", "age",
"salary"])

# Fill nulls
df_filled = df_nulls.fillna({"age": 0, "salary": 0})
df_filled_specific = df_nulls.fillna("Unknown", subset=["name"])

df_filled.show()
```

tom

Impose Schema

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DateType

# Define custom schema
custom_schema = StructType([
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("salary", IntegerType(), True),
    StructField("hire_date", DateType(), True)
])

# Read with custom schema
df_schema = spark.read.schema(custom_schema).csv("data/sample.csv")
df_schema.printSchema()
```

Write Modes

```
# Different write modes
df.write.mode("overwrite").parquet("output/overwrite_mode")
df.write.mode("append").parquet("output/append_mode")
df.write.mode("ignore").parquet("output/ignore_mode")
df.write.mode("error").parquet("output/error_mode")

print("Data written with different modes")
```

end

Date Functions

```
from pyspark.sql.functions import current_date, datediff, months_between, add_months, date_format

# Date operations
df_dates = df.withColumn("current_date", current_date()) \
    .withColumn("days_since_hire", datediff(current_date(), lit("2020-01-01"))) \
    .withColumn("months_diff", months_between(current_date(), lit("2020-01-01"))) \
    .withColumn("next_review", add_months(current_date(), 6)) \
```

```
        .withColumn("formatted_date", date_format(current_date(),
"yyyy-MM"))

df_dates.show()
```

Common Categories of Built-in Functions

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# Create Spark session
spark = SparkSession.builder \
    .appName("UDFs Demo") \
    .getOrCreate()

# Sample data
data = [("John", 25, 50000), ("Alice", 30, 60000), ("Bob", 35, 75000)]
df = spark.createDataFrame(data, ["name", "age", "salary"])

# String functions
df_string = df.withColumn("upper_name", upper(col("name"))) \
    .withColumn("name_length", length(col("name")))

# Numeric functions
df_numeric = df.withColumn("salary_in_k", col("salary") / 1000) \
    .withColumn("age_squared", pow(col("age"), 2))

# Date functions
df_date = df.withColumn("current_date", current_date()) \
    .withColumn("current_timestamp", current_timestamp())

# Conditional functions
df_conditional = df.withColumn("age_group",
                                when(col("age") < 30, "Young")
                                .when(col("age") < 40, "Middle-aged")
                                .otherwise("Senior"))

# Aggregate functions
df_agg = df.agg(
    avg("salary").alias("avg_salary"),
    max("age").alias("max_age"),
    count("*").alias("total_records")
```

```
)  
  
df.show()  
df_agg.show()
```

Regular UDFs (User Defined Functions)

Simple Scalar UDFs

```
# Register and use regular Python UDF  
def calculate_bonus(salary, multiplier=0.1):  
    """Calculate bonus as percentage of salary"""  
    return float(salary * multiplier)  
  
# Register UDF  
calculate_bonus_udf = udf(calculate_bonus, FloatType())  
  
# Use UDF  
df_with_bonus = df.withColumn("bonus", calculate_bonus_udf(col("salary"),  
lit(0.15)))  
df_with_bonus.show()  
  
# UDF with multiple parameters  
def salary_category(salary, age):  
    if salary > 70000:  
        return "High"  
    elif salary > 55000 and age > 25:  
        return "Medium"  
    else:  
        return "Low"  
  
salary_category_udf = udf(salary_category, StringType())  
df_with_category = df.withColumn("category",  
salary_category_udf(col("salary"), col("age")))  
df_with_category.show()
```

Working with ArrayType Columns in PySpark

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# Create Spark session
spark = SparkSession.builder \
    .appName("ArrayType Columns") \
    .getOrCreate()

# Sample data with arrays
data = [
    (1, "John", ["Math", "Physics", "Chemistry"], [85, 90, 78]),
    (2, "Alice", ["English", "History"], [92, 88]),
    (3, "Bob", ["Math", "Computer Science", "Physics", "Chemistry"], [75,
85, 82, 79]),
    (4, "Charlie", ["Art", "Music"], [95, 91]),
    (5, "David", [], []), # Empty arrays
    (6, "Eve", None, None) # Null arrays
]

schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("subjects", ArrayType(StringType())),
    StructField("marks", ArrayType(IntegerType()))
])

df = spark.createDataFrame(data, schema)
df.show(truncate=False)
df.printSchema()

```

Array Functions & Transformations

Array Size and Length

```

# Get array size/length
df_size = df.select(
    col("id"),
    col("name"),
    col("subjects"),

```

```

    size(col("subjects")).alias("num_subjects"),
    cardinality(col("subjects")).alias("num_subjects_alt"),  # Same as size
    col("marks"),
    size(col("marks")).alias("num_marks")
)
df_size.show()

# Check if array is empty
df_empty = df.select(
    col("name"),
    col("subjects"),
    size(col("subjects")).alias("size"),
    array_size(col("subjects")).alias("size_alt"),  # Another alternative
    col("subjects").isNull().alias("is_null"),
    col("subjects").isNotNull().alias("is_not_null"),
    array_contains(col("subjects"), "Math").alias("has_math"),
    array_position(col("subjects"), "Math").alias("math_position")  #
1-based position
)
df_empty.show()

```

Array Manipulation

```

# Concatenate arrays
df_concat = df.select(
    col("id"),
    col("name"),
    col("subjects"),
    array_union(
        col("subjects"),
        array(lit("PE"), lit("Drama"))
    ).alias("subjects_with_extras"),
    array_intersect(
        col("subjects"),
        array(lit("Math"), lit("Physics"), lit("Biology"))
    ).alias("common_subjects"),
    array_except(
        col("subjects"),
        array(lit("Math"), lit("Chemistry"))
    ).alias("subjects_without_math_chem")
)

```

```

display(df_concat)

# Remove duplicates from array
df_distinct = df.select(
    col("name"),
    col("subjects"),
    array_distinct(col("subjects")).alias("unique_subjects")
)
display(df_distinct)

# Sort array elements (descending not supported directly)
df_sorted = df.select(
    col("name"),
    col("subjects"),
    array_sort(col("subjects")).alias("sorted_subjects_asc")
)
display(df_sorted)

# Reverse array
df_reversed = df.select(
    col("name"),
    col("subjects"),
    reverse(col("subjects")).alias("reversed_subjects")
)
display(df_reversed)

```

Exploding Arrays

Explode (Creates New Rows)

```

# Using arrays_zip to explode multiple arrays together
df_zipped = df.select(
    col("id"),
    col("name"),
    explode(arrays_zip(col("subjects"), col("marks"))).alias("zipped")
).select(
    col("id"),
    col("name"),
    col("zipped")["subjects"].alias("subject"),
    col("zipped")["marks"].alias("mark")
)

```

```
display(df_zipped)
```

Inline Explosion (for Arrays of Structs)

```
# Create array of structs
```

```
struct_data = [  
    (1, "John", [("Math", 85), ("Physics", 90), ("Chemistry", 78)]),  
    (2, "Alice", [("English", 92), ("History", 88)]),  
    (3, "Bob", [("Math", 75), ("Computer Science", 85)])  
]
```

```
struct_schema = StructType([  
    StructField("id", IntegerType()),  
    StructField("name", StringType()),  
    StructField("subject_marks", ArrayType(  
        StructType([  
            StructField("subject", StringType()),  
            StructField("mark", IntegerType())  
        ]) )  
    ))  
)
```

```
struct_df = spark.createDataFrame(struct_data, struct_schema)  
struct_df.show(truncate=False)
```

```
# Use inline to explode array of structs
```

```
inline_df = struct_df.select(  
    col("id"),  
    col("name"),  
    inline(col("subject_marks"))  
)  
inline_df.show()
```

View & Understand Data

1. df.show()

Displays rows of the DataFrame (default 20).

```
df.show()
```

2. df.printSchema()

Shows column names and data types in tree format.

```
df.printSchema()
```

3. df.count()

Returns total number of rows.

```
df.count()
```

4. df.columns

Returns list of column names.

```
df.columns
```

5. df.dtypes

Shows column names with data types.

```
df.dtypes
```

6. df.head()

Returns first row.

```
df.head()
```

7. df.take(5)

Returns first 5 rows as a list.

```
df.take(5)
```

8. df.limit(10)

Restricts DataFrame to first 10 rows.

```
df.limit(10).show()
```

9. Read CSV

Reads CSV file into DataFrame.

```
spark.read.csv("path", header=True)
```

10. Read Parquet

Reads Parquet (columnar format, faster).

```
spark.read.parquet("path")
```

Select & Filter Data

11. Select one column

```
df.select("name")
```

Selects only name column.

12. Select multiple columns

```
df.select("name", "salary")
```

13. selectExpr

Use SQL expressions.

```
df.selectExpr("salary * 2 as double_salary")
```

14. Filter using condition

```
df.filter(df.salary > 50000)
```

15. where (same as filter)

```
df.where(df.age >= 30)
```

16. Filter using SQL string

```
df.filter("salary > 50000")
```

17. distinct()

Removes duplicate rows.

```
df.distinct()
```

18. dropDuplicates()

Removes duplicate rows (same as distinct).

```
df.dropDuplicates()
```

19. Drop column

```
df.drop("temp_col")
```

20. Filter by value

```
df.filter(df.country == "India")
```

Column Operations

21. withColumn

Create or modify a column.

```
df.withColumn("bonus", df.salary * 0.1)
```

22. Rename column

```
df.withColumnRenamed("dept", "department")
```

23. dropna()

Removes rows with null values.

```
df.dropna()
```

24. fillna()

Replace null values with 0.

```
df.fillna(0)
```

25. Fill null for specific column

```
df.fillna({"salary": 0})
```

26. Add current date

```
df.withColumn("today", current_date())
```

27. lit() – constant column

```
df.withColumn("flag", lit(1))
```

Adds constant value to all rows.

28. Cast data type

```
df.withColumn("age", df.age.cast("int"))
```

29. Alias column

```
df.select(col("salary").alias("income"))
```

30. Uppercase column

```
df.withColumn("upper_name", upper(col("name")))
```

31–40: Sorting & Aggregation

31. Sort ascending

```
df.orderBy("salary")
```

32. Sort descending

```
df.orderBy(df.salary.desc())
```

33. Group and count

```
df.groupBy("dept").count()
```

34. Group and average

```
df.groupBy("dept").avg("salary")
```

35. Group and sum

```
df.groupBy("dept").sum("salary")
```

36. Aggregate max

```
df.agg({"salary": "max"})
```

37. Multiple aggregations

```
df.groupBy("dept").agg(avg("salary"))
```

38. Count rows

```
df.groupBy("dept").agg(count("*"))
```

39. Minimum value

```
df.groupBy("dept").agg(min("salary"))
```

40. Maximum value

```
df.groupBy("dept").agg(max("salary"))
```

Join, Union & Performance

41. Inner join

```
df1.join(df2, "id")
```

42. Left join

```
df1.join(df2, "id", "left")
```

43. Join with condition

```
df1.join(df2, df1.id == df2.id)
```

44. Union

```
df.union(df2)
```

Same schema required.

45. Union by column name

```
df.unionByName(df2)
```

46. Write CSV

```
df.write.csv("output/path")
```

47. Write Parquet

```
df.write.parquet("output/path")
```

48. Save as table

```
df.write.mode("overwrite").saveAsTable("emp")
```

49. Cache DataFrame

```
df.cache()
```

Improves performance for repeated use.

50. Execution plan

```
df.explain()
```

Shows logical & physical execution plan

dbutils (Databricks Utilities)

dbutils is a **built-in helper library** used to interact with:

- DBFS
- Secrets
- Widgets
- Notebooks
- Jobs

Major dbutils Modules

dbutils.fs
dbutils.widgets
dbutils.secrets
dbutils.notebook
dbutils.jobs

DBFS (Databricks File System)

What it is

DBFS is a **distributed file system abstraction** over cloud storage (ADLS / S3 / GCS) used by Databricks.
Think of DBFS as **Databricks' gateway to cloud storage**.

Real Use Case

- Store raw, processed, curated data
- Temporary files during ETL
- Logging & checkpoints

Secrets (Secure Credential Management)

What it is

Secrets allow **secure storage of passwords, keys, tokens** without hardcoding.

➡ Values are **masked in logs**.

Real Use Case

- ADLS access keys
- SQL passwords
- API tokens
- Service principal secrets

Widgets (Notebook Parameters)

What it is

Widgets are **input parameters** for notebooks, used heavily in **Jobs & pipelines**.

Widget Types

```
dbutils.widgets.text("env", "dev")
```

```
dbutils.widgets.dropdown("country", "IN", ["IN", "US"])
```

```
dbutils.widgets.multiselect("source", "adf", ["adf","kafka"])
```

```
dbutils.widgets.combobox("year", "2025", ["2024","2025"])
```

Read Widget Values

```
env = dbutils.widgets.get("env")
```

Real Use Case

- Same notebook for dev / test / prod
- Date-based incremental loads
- Reusable ETL pipelines

Notebooks (Execution & Orchestration)

What it is

Notebooks are **executable units** containing code (Python, SQL, Scala).

They can:

- Call other notebooks
- Return status
- Be reused like functions

Real Use Case

- Modular ETL (ingestion → transform → load)
- Reusable logic blocks
- Workflow orchestration

Jobs (Databricks Workflows)

What it is

Jobs allow **scheduled and triggered execution** of notebooks, JARs, or pipelines.

Job Features

- Scheduling (cron)
 - Retry & failure handling
 - Parameter passing
 - Task dependencies
 - Job clusters
-

One Liners

- **DBFS** → Storage abstraction layer
- **Secrets** → Secure credential store
- **Widgets** → Notebook parameters
- **Notebooks** → Execution units
- **Jobs** → Orchestration & scheduling

Dbutils use cases

Step 1: List files in DBFS root

```
print("Listing files in DBFS root:")
files = dbutils.fs.ls("/")
for file in files:
    print(f"• {file.name} (Size: {file.size} bytes)")
```

Step 2: List files in a specific folder

```
try:
    print("\nFiles in /databricks-datasets:")
    datasets = dbutils.fs.ls("/databricks-datasets")
    for ds in datasets[:20]: # Show first 20
        print(f"  - {ds.name}")
except:
    print("Folder not found, creating sample structure...")
    # Create a sample file
    dbutils.fs.put("/sample.txt", "Hello, Databricks!")
```

Step 1: Create a text file

```
dbutils.fs.put("/tmp/my_first_file.txt",
               "This is my first file created with dbutils!\nLine 2\nLine 3")
```

Step 2: Read the file

```
print("Reading file content:")
content = dbutils.fs.head("/tmp/my_first_file.txt", 500) # Read first 500 bytes
print(content)
```

Step 3: Copy the file

```
dbutils.fs.cp("/tmp/my_first_file.txt", "/tmp/copied_file.txt")
```

Step 4: delete the file

```
dbutils.fs.rm("/tmp/my_first_file.txt")
dbutils.fs.rm("/tmp/copied_file.txt")
print("✓ Files deleted successfully!")
```

Step 1: Create directories

```
dbutils.fs.mkdirs("/tmp/my_project/data/raw")
dbutils.fs.mkdirs("/tmp/my_project/data/processed")
dbutils.fs.mkdirs("/tmp/my_project/logs")
```

Step 2: Create sample files

```
for day in range(1, 4):
    dbutils.fs.put(f"/tmp/my_project/data/raw/day{day}.csv",
                  f"id,name,value\n1,item{day},100\n2,item{day},200")
```

Step 3: List created structure

```
print("Project structure:")
for item in dbutils.fs.ls("/tmp/my_project"):
    print(f"📁 {item.name}")
    if item.name == "data":
        for subitem in dbutils.fs.ls("/tmp/my_project/data"):
            print(f"└─ 📁 {subitem.name}")
            if subitem.name == "raw":
                for file in dbutils.fs.ls("/tmp/my_project/data/raw"):
                    print(f"└─ 📄 {file.name}")
```

Mount Azure Storage (Blob)

Step 1: Define mount point (where it will appear in DBFS)

```
mount_point = "/mnt/myblobstorage"
```

```

# Step 2: Unmount if already mounted (clean start)
try:
    dbutils.fs.unmount(mount_point)
    print(f"✓ Unmounted existing mount at {mount_point}")
except:
    print(f"No existing mount at {mount_point}")

# Step 3: Mount Azure Blob Storage
# Replace with your actual storage details
storage_account = "yourstorageaccount"
container = "yourcontainer"
sas_token = "?sv=..." # Your SAS token

if not any(mount.mountPoint == mount_point for mount in dbutils.fs.mounts()):
    dbutils.fs.mount(
        source=f"wasbs://{container}@{storage_account}.blob.core.windows.net",
        mount_point=mount_point,
        extra_configs={
            f"fs.azure.sas.{container}.{storage_account}.blob.core.windows.net":
sas_token
        }
    )
    print(f"✓ Storage mounted at {mount_point}")
else:
    print(f"✓ Storage already mounted at {mount_point}")

# Step 4: List mounts
print("\nCurrent mounts:")
for mount in dbutils.fs.mounts():
    print(f"    • {mount.mountPoint} → {mount.source}")

```

Basic Widgets for Interactive Notebooks

```

# Step 1: Remove any existing widgets
dbutils.widgets.removeAll()

# Step 2: Create different types of widgets
print("Creating widgets...")

# Text widget
dbutils.widgets.text("username", "databricks_user", "Enter Username")

# Dropdown widget
dbutils.widgets.dropdown("country", "USA",

```

```

        ["USA", "Canada", "UK", "Australia", "India"],
        "Select Country")

# Multiselect widget
dbutils.widgets.multiselect("interests", "Python",
                             ["Python", "SQL", "Machine Learning", "Spark", "Data
Engineering"],
                             "Select Interests")

# Step 3: Get widget values
print("\nGetting widget values:")
username = dbutils.widgets.get("username")
country = dbutils.widgets.get("country")
interests = dbutils.widgets.get("interests")

print(f"Username: {username}")
print(f"Country: {country}")
print(f"Interests: {interests}")

# Step 4: Use widget values in code
print(f"\nWelcome {username} from {country}!")
print(f"Your interests: {interests}")

```

File Operations with Progress

```

# Step 1: Create multiple test files
print("Creating test files...")
for i in range(5):
    content = f"File {i}\n" + "\n".join([f"Line {j}: Data {i}{j}" for j in
range(10)])
    dbutils.fs.put(f"/tmp/testfiles/file_{i}.txt", content)
    print(f"    Created file_{i}.txt")

# Step 2: Count total lines in all files

print("\nCounting lines in all files...")
total_lines = 0
files = dbutils.fs.ls("/tmp/testfiles/")

for file_info in files:
    if file_info.name.endswith(".txt"):
        # Read entire file
        content = dbutils.fs.head(file_info.path, 10000)
        lines = content.count('\n')
        total_lines += lines

```

```

        print(f"    {file_info.name}: {lines} lines")

print(f"\n Total: {len(files)} files, {total_lines} lines")

# Step 3: Move files to archive
print("\nMoving files to archive folder...")
dbutils.fs.mkdirs("/tmp/testfiles/archive/")

for file_info in files:
    if file_info.name.endswith(".txt"):
        source = file_info.path
        destination = f"/tmp/testfiles/archive/{file_info.name}"
        dbutils.fs.mv(source, destination)
        print(f"    Moved: {file_info.name}")

# Step 4: Cleanup
print("\nCleaning up...")
dbutils.fs.rm("/tmp/testfiles/", recurse=True)
print("✓ Cleanup completed!")

```

```

print("=== Multiple File Operations ===")

# Create multiple department files
departments = {
    "engineering": """id,name,project,salary
1,John Doe,Project A,75000
2,Alice Brown,Project B,80000
3,David Lee,Project C,72000""",

    "marketing": """id,name,campaign,salary
4,Jane Smith,Spring Campaign,65000
5,Charlie Wilson,Summer Campaign,60000""",

    "sales": """id,name,region,sales_target
6,Bob Johnson,North,1000000
7,Sarah Davis,South,1200000
8,Michael Chen,West,950000"""
}

# Save each department file
for dept, content in departments.items():
    dbutils.fs.put(f"/tmp/{dept}_data.csv", content)
    print(f"✓ Created {dept}_data.csv")

```

```
# List all CSV files in /tmp
print("\n=== All CSV files in /tmp ===")
all_files = dbutils.fs.ls("/tmp")
csv_files = [f for f in all_files if f.name.endswith('.csv')]

for file in csv_files:
    print(f"• {file.name} - {file.size} bytes")

    # Read first line of each file
    content = dbutils.fs.head(file.path, 200)
    print(f"    Preview: {content[:100]}...")
    print()
```

Basic Widgets for Data Filtering

1. Text widget for minimum value

```
dbutils.widgets.text(
    name="min_salary",
    defaultValue="50000",
    label="Minimum Salary"
)
```

2. Dropdown for department selection

```
dbutils.widgets.dropdown(
    name="department",
    defaultValue="All",
    choices=["All", "Engineering", "Marketing", "Sales", "Finance"],
    label="Select Department"
)
```

3. Multiselect for status

```
dbutils.widgets.multiselect(
    name="status",
    defaultValue="Active",
    choices=["Active", "On Leave", "Terminated", "Retired"],
    label="Employee Status"
)
```

```

# 4. Combobox for location (can select or type)
dbutils.widgets.combobox(
    name="location",
    defaultValue="New York",
    choices=["New York", "London", "Tokyo", "Singapore", "Sydney"],
    label="Office Location"
)

print("✓ Widgets created successfully!")
print("Change widget values above and re-run the cell below")

# ===== USE THE WIDGETS =====
print("\n=== Using Widget Values ===")

# Get widget values
min_salary = dbutils.widgets.get("min_salary")
department = dbutils.widgets.get("department")
status_list = dbutils.widgets.get("status")
location = dbutils.widgets.get("location")

print(f" Filter Criteria:")
print(f"    • Minimum Salary: ${min_salary}")
print(f"    • Department: {department}")
print(f"    • Status: {status_list}")
print(f"    • Location: {location}")

# Sample data to demonstrate filtering
sample_data = [
    (1, "John Doe", "Engineering", 75000, "Active", "New York"),
    (2, "Jane Smith", "Marketing", 65000, "Active", "London"),
    (3, "Bob Johnson", "Sales", 45000, "Active", "Tokyo"),
    (4, "Alice Brown", "Engineering", 80000, "On Leave", "New York"),
    (5, "Charlie Wilson", "Finance", 55000, "Active", "Singapore")
]

# Create DataFrame
columns = ["id", "name", "department", "salary", "status", "location"]
df = spark.createDataFrame(sample_data, columns)

# Apply filters based on widgets
filtered_df = df.filter(f"salary >= {min_salary}")

if department != "All":
    filtered_df = filtered_df.filter(f"department = '{department}'")

# For multiselect, create OR condition

```

```

if status_list:
    statuses = status_list.split(",")
    status_condition = " OR ".join([f"status = '{s.strip()}'" for s in statuses])
    filtered_df = filtered_df.filter(status_condition)

if location:
    filtered_df = filtered_df.filter(f"location = '{location}'")

print(f"\n✅ Filter Results: {filtered_df.count()} employees found")
display(filtered_df)

```

JSON

```

# Import necessary libraries
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType, DoubleType, DateType
from pyspark.sql.functions import lit, col, current_date, to_date
import json
import pandas as pd
from datetime import datetime

# Initialize Spark session
spark = SparkSession.builder \
    .appName("DBFS_JSON_Management") \
    .config("spark.sql.legacy.timeParserPolicy", "LEGACY") \
    .getOrCreate()

# Display DBFS root

```

```
display(dbutils.fs.ls("/"))
```

Create sample data as Python dictionary

```
sample_data = [  
    {"employee_id": 1, "name": "John Doe", "department": "IT", "salary":  
75000.50, "join_date": "2023-01-15"},  
    {"employee_id": 2, "name": "Jane Smith", "department": "HR", "salary":  
65000.00, "join_date": "2022-06-01"},  
    {"employee_id": 3, "name": "Bob Johnson", "department": "Finance", "salary":  
82000.75, "join_date": "2021-03-10"},  
    {"employee_id": 4, "name": "Alice Brown", "department": "IT", "salary":  
72000.25, "join_date": "2023-08-22"},  
    {"employee_id": 5, "name": "Charlie Wilson", "department": "Marketing",  
"salary": 68000.00, "join_date": "2022-11-30"}  
]
```

Convert to JSON string

```
json_data = json.dumps(sample_data)
```

Write to DBFS

```
dbutils.fs.put("/tmp/sample_employees.json", json_data, overwrite=True)  
print("JSON file created successfully!")
```

Verify file creation

```
display(dbutils.fs.ls("/tmp/sample_employees.json"))
```

Create Spark DataFrame

```
data = [  
    (101, "Product A", 299.99, 50, "Electronics"),  
    (102, "Product B", 149.99, 120, "Home"),  
    (103, "Product C", 89.99, 200, "Kitchen"),  
    (104, "Product D", 599.99, 25, "Electronics"),  
    (105, "Product E", 39.99, 300, "Clothing")  
]
```

```
schema = ["product_id", "product_name", "price", "stock", "category"]  
products_df = spark.createDataFrame(data, schema=schema)
```

Add timestamp column

```
products_df = products_df.withColumn("last_updated", current_date())
```

Write DataFrame as JSON to DBFS

```
json_path = "/tmp/products/products_data.json"  
products_df.write \
```

```
.mode("overwrite") \
.json(json_path)

print(f"DataFrame saved as JSON to: {json_path}")
```

Create nested JSON structure

```
complex_data = [
    {
        "customer_id": "C001",
        "name": {"first": "John", "last": "Doe"},
        "contact": {
            "email": "john.doe@email.com",
            "phone": "+1234567890"
        },
        "orders": [
            {"order_id": "0001", "amount": 150.50, "date": "2024-01-15"},
            {"order_id": "0002", "amount": 89.99, "date": "2024-02-20"}
        ],
        "preferences": {
            "newsletter": True,
            "promotions": False
        }
    },
    {
        "customer_id": "C002",
        "name": {"first": "Jane", "last": "Smith"},
        "contact": {
            "email": "jane.smith@email.com",
            "phone": "+0987654321"
        },
        "orders": [
            {"order_id": "0003", "amount": 299.99, "date": "2024-03-05"}
        ],
        "preferences": {
            "newsletter": True,
            "promotions": True
        }
    }
]
```

Write nested JSON to DBFS

```
nested_json_path = "/tmp/customers/customers_nested.json"
dbutils.fs.put(nested_json_path, json.dumps(complex_data), overwrite=True)
```

Read JSON file

```
employees_df = spark.read.json("/tmp/sample_employees.json")
employees_df.show()
employees_df.printSchema()
```

Read with schema inference

```
products_df = spark.read \
    .option("inferSchema", "true") \
    .json("/tmp/products/products_data.json")
products_df.show()
```

Define schema explicitly

```
employee_schema = StructType([
    StructField("employee_id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("department", StringType(), True),
    StructField("salary", DoubleType(), True),
    StructField("join_date", DateType(), True)
])
```

```
employees_df = spark.read \
    .schema(employee_schema) \
    .json("/tmp/sample_employees.json")
```

```
employees_df.show()
employees_df.printSchema()
```

Read all JSON files in a directory

```
all_products_df = spark.read.json("/tmp/products/*.json")
```

Read with specific options

```
multi_json_df = spark.read \
    .option("multiLine", "true") \
    .option("mode", "PERMISSIVE") \
    .json("/tmp/customers/*.json")
```

List files in directory

```

display(dbutils.fs.ls("/tmp/"))

# Check if file exists
def file_exists(path):
    try:
        dbutils.fs.ls(path)
        return True
    except Exception:
        return False

print(f"File exists: {file_exists('/tmp/sample_employees.json')}")

# Copy JSON file
dbutils.fs.cp("/tmp/sample_employees.json", "/tmp/backup/employees_backup.json")

# Move/Rename JSON file
dbutils.fs.mv("/tmp/sample_employees.json", "/tmp/archived/old_employees.json")

# Delete JSON file
dbutils.fs.rm("/tmp/archived/old_employees.json")

```

```

# New data to append
new_employees = [
    {"employee_id": 6, "name": "David Lee", "department": "Sales", "salary": 58000.00,
    "join_date": "2024-01-10"},
    {"employee_id": 7, "name": "Emily Chen", "department": "IT", "salary": 81000.00,
    "join_date": "2024-02-15"}
]

new_df = spark.createDataFrame(new_employees)

# Append to existing JSON directory
new_df.write \
    .mode("append") \
    .json("/dbfs/json_data/raw/employees.json")

# Verify append
appended_df = spark.read.json("/dbfs/json_data/raw/employees.json")
print(f"Total records after append: {appended_df.count()}")
will not work in serverless

```

Partitioned JSON Storage

```

# Create partitioned JSON structure
sales_data = [

```

```

    (1, "Product A", 100.50, "2024-01-15", "North"),
    (2, "Product B", 200.75, "2024-01-15", "South"),
    (3, "Product A", 150.00, "2024-01-16", "North"),
    (4, "Product C", 75.25, "2024-01-16", "East"),
    (5, "Product B", 225.00, "2024-01-17", "West")
]

sales_df = spark.createDataFrame(sales_data, ["sale_id", "product", "amount", "date",
"region"])

# Write with partitioning
sales_df.write \
    .mode("overwrite") \
    .partitionBy("date", "region") \
    .json("/dbfs/json_data/partitioned/sales")

# Read partitioned data efficiently
partitioned_sales = spark.read.json("/dbfs/json_data/partitioned/sales")
print(f"Partitions: {partitioned_sales.rdd.getNumPartitions()}")

```

Complete ETL Pipeline Example

```

# Import necessary libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import lit, current_date, col
from datetime import datetime, timedelta

# Initialize Spark session
spark = SparkSession.builder \
    .appName("JSONManager") \
    .getOrCreate()

# Make dbutils available (for Databricks environment)
try:
    from pyspark.dbutils import DBUtils
    dbutils = DBUtils(spark)
except:
    # For non-Databricks environments, create a mock dbutils
    class MockDBUtils:
        def fs(self):
            class FS:
                def ls(self, path):
                    return []
                def mv(self, src, dest):
                    print(f"Mock: Moving {src} to {dest}")
                def rm(self, path, recurse=False):
                    print(f"Mock: Removing {path}")
            return FS()
    dbutils = MockDBUtils()

```

```

class JSONManager:
    def __init__(self, base_path="/dbfs/json_data"):
        self.base_path = base_path
        self.spark = spark # Use the global spark session

    def create_daily_snapshot(self, data, entity_name):
        """Create daily JSON snapshot with timestamp"""
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        df = self.spark.createDataFrame(data)

        # Add metadata columns
        df = df.withColumn("load_timestamp", lit(timestamp))
        df = df.withColumn("load_date", current_date())

        # Define output path
        output_dir = f"{self.base_path}/snapshots/{entity_name}/date={timestamp[:8]}"

        # Write JSON
        df.coalesce(1).write \
            .mode("overwrite") \
            .json(f"{output_dir}/snapshot_{timestamp}.json")

        print(f"Snapshot created: {output_dir}")
        return output_dir

    def archive_old_files(self, path, days_to_keep=30):
        """Archive files older than specified days"""
        cutoff_date = datetime.now() - timedelta(days=days_to_keep)

        files = dbutils.fs.ls(path)
        for file in files:
            # Check if modification time is older than cutoff
            if file.modificationTime/1000 < cutoff_date.timestamp():
                archive_path = file.path.replace("/snapshots/", "/archive/")
                dbutils.fs.mv(file.path, archive_path)
                print(f"Archived: {file.name}")

    def merge_json_files(self, source_pattern, output_path):
        """Merge multiple JSON files into one"""
        combined_df = self.spark.read.json(source_pattern)

        # Write as single JSON file
        combined_df.coalesce(1).write \
            .mode("overwrite") \
            .json(output_path)

        return combined_df.count()

    # Additional utility methods
    def list_json_files(self, directory):
        """List all JSON files in a directory"""
        try:
            files = dbutils.fs.ls(directory)
            json_files = [f for f in files if f.path.endswith('.json')]
            return json_files
        except Exception as e:

```

```

        print(f"Error listing files: {e}")
        return []

def validate_json_schema(self, file_path):
    """Validate JSON file schema"""
    try:
        df = self.spark.read.json(file_path)
        return {
            "is_valid": True,
            "schema": df.schema,
            "row_count": df.count(),
            "columns": df.columns
        }
    except Exception as e:
        return {
            "is_valid": False,
            "error": str(e)
        }

def create_backup(self, source_path, backup_suffix="_backup"):
    """Create backup of JSON files"""
    import time
    timestamp = int(time.time())
    backup_path = f"{source_path}_{timestamp}{backup_suffix}"

    # In production, you would use dbutils.fs.cp
    print(f"Backup created: {backup_path}")
    return backup_path

# Usage example
if __name__ == "__main__":
    # Initialize JSONManager
    manager = JSONManager(base_path="/tmp/json_data") # Using /tmp for testing

    # Create daily snapshot
    sample_sales = [
        {"sale_id": 1, "amount": 100, "product": "A"},
        {"sale_id": 2, "amount": 200, "product": "B"},
        {"sale_id": 3, "amount": 150, "product": "C"}
    ]

    print("Creating daily snapshot...")
    snapshot_path = manager.create_daily_snapshot(sample_sales, "sales")

    # Validate the created JSON
    print("\nValidating JSON schema...")
    validation_result = manager.validate_json_schema(f"{snapshot_path}/snapshot_*.json")
    print(f"Validation result: {validation_result}")

    # List JSON files
    print("\nListing JSON files...")
    json_files = manager.list_json_files("/tmp/json_data/snapshots/sales/")
    for file in json_files:
        print(f"  - {file.name} ({file.size} bytes)")

    # Merge files (simulated - would need actual files)
    print("\nSimulating merge operation...")

```

```

# weekly_count = manager.merge_json_files(
#     "/tmp/json_data/snapshots/sales/date=2024*/*.json",
#     "/tmp/json_data/reports/sales_weekly.json"
# )
# print(f"Merged {weekly_count} records into weekly report")

# Clean up
spark.stop()
print("\nProcess completed!")

```

NoteBOOK to create job

```

# Databricks notebook source
# DBTITLE 1,JSON File Manager - Complete Solution

# COMMAND -----
# DBTITLE 1,Import Libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import lit, current_date
import json
from datetime import datetime, timedelta
import sys

# COMMAND -----
# DBTITLE 1,Initialize Spark Session
# Initialize Spark session
spark = SparkSession.builder \
    .appName("JSON_File_Manager") \
    .getOrCreate()

print(f"PySpark version: {spark.version}")
print(f"Python version: {sys.version}")

# COMMAND -----
# DBTITLE 1,Configuration
class Config:
    def __init__(self):
        # Base paths
        self.base_path = "/dbfs/json_data"
        self.raw_path = f"{self.base_path}/raw"
        self.processed_path = f"{self.base_path}/processed"
        self.archive_path = f"{self.base_path}/archive"
        self.reports_path = f"{self.base_path}/reports"

        # Create directories
        for path in [self.raw_path, self.processed_path, self.archive_path,
self.reports_path]:

```

```

        dbutils.fs.mkdirs(path)
        print(f"✓ Directory ready: {path}")

# Default settings
self.days_to_keep = 30
self.compression = "gzip"

config = Config()

# COMMAND -----
# DBTITLE 1,JSON Manager Functions
def process_json_files(entity_name, input_path=None):
    """
    Process JSON files for a specific entity
    """
    if input_path is None:
        input_path = f"{config.raw_path}/{entity_name}/*.json"

    print(f"Processing {entity_name} from: {input_path}")

    try:
        # Read JSON files
        df = spark.read.json(input_path)
        record_count = df.count()
        print(f"  Read {record_count} records")

        if record_count == 0:
            print(f"  No data found for {entity_name}")
            return None

        # Add metadata
        df = df.withColumn("_load_date", current_date())
        df = df.withColumn("_load_timestamp", lit(datetime.now().isoformat()))

        # Save with date partitioning
        today = datetime.now()
        output_path =
f"{config.processed_path}/{entity_name}/year={today.year}/month={today.month:02d}/day={to
day.day:02d}"

        df.write \
            .mode("overwrite") \
            .partitionBy("_load_date") \
            .option("compression", config.compression) \
            .json(output_path)

        print(f"  Saved to: {output_path}")
        return {"entity": entity_name, "records": record_count, "path": output_path}

    except Exception as e:
        print(f"  Error processing {entity_name}: {str(e)}")

```

```

        return None

def create_snapshot(entity_name, days_back=1):
    """
    Create snapshot from processed data
    """
    snapshot_date = datetime.now() - timedelta(days=days_back)
    date_str = snapshot_date.strftime("%Y-%m-%d")

    print(f"Creating snapshot for {entity_name} (data from {date_str})")

    try:
        # Read data for the specific date
        df =
spark.read.json(f"{config.processed_path}/{entity_name}/*/_load_date={date_str}/*.json")

        if df.count() == 0:
            print(f" No data found for {entity_name} on {date_str}")
            return None

        # Save snapshot
        snapshot_path =
f"{config.reports_path}/snapshots/{entity_name}_{datetime.now().strftime('%Y%m%d')}.json"

        df.coalesce(1).write \
            .mode("overwrite") \
            .json(snapshot_path)

        print(f" Snapshot saved: {snapshot_path}")
        return {"entity": entity_name, "snapshot_path": snapshot_path, "records":
df.count()}

    except Exception as e:
        print(f" Error creating snapshot: {str(e)}")
        return None

def archive_old_data(days_threshold=30):
    """
    Archive data older than specified days
    """
    print(f"Archiving data older than {days_threshold} days")

    cutoff_date = datetime.now() - timedelta(days=days_threshold)
    archived_count = 0

    try:
        # List all processed entities
        entities = dbutils.fs.ls(config.processed_path)

        for entity in entities:
            if entity.isDir():

```

```

        # Check year directories
        years = dbutils.fs.ls(entity.path)
        for year in years:
            if year.isDir():
                year_num = int(year.name.replace("year=", "").rstrip("/"))

                # Check month directories
                months = dbutils.fs.ls(year.path)
                for month in months:
                    if month.isDir():
                        month_num = int(month.name.replace("month=",
"").rstrip("/"))

                        # Check if this month is old enough to archive
                        dir_date = datetime(year_num, month_num, 1)
                        if dir_date < cutoff_date:
                            # Archive this directory
                            archive_path = month.path.replace("/processed/",
"/archive/")

                            dbutils.fs.mv(month.path, archive_path, recurse=True)
                            archived_count += 1
                            print(f"  Archived: {month.path}")

except Exception as e:
    print(f"  Error during archiving: {str(e)}")

print(f"  Archived {archived_count} directories")
return archived_count

def get_file_stats():
    """
    Get statistics about JSON files
    """
    print("Getting file statistics...")

    stats = {}
    directories = {
        "raw": config.raw_path,
        "processed": config.processed_path,
        "archive": config.archive_path,
        "reports": config.reports_path
    }

    for name, path in directories.items():
        try:
            files = dbutils.fs.ls(path)
            total_size = sum(f.size for f in files)

            stats[name] = {
                "files": len(files),
                "size_mb": round(total_size / (1024**2), 2),

```

```

        "sample": [f.name for f in files[:3]] # First 3 files
    }
except Exception as e:
    stats[name] = {"error": str(e)}

return stats

def create_test_data(entity_name="test", record_count=100):
    """
    Create test JSON data
    """
    print(f"Creating test data for {entity_name} ({record_count} records)")

    test_data = []
    for i in range(1, record_count + 1):
        test_data.append({
            "id": i,
            "name": f"Item_{i}",
            "value": i * 10,
            "category": "Test",
            "timestamp": datetime.now().isoformat()
        })

    # Save as JSON
    output_path =
f"{config.raw_path}/{entity_name}/test_data_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"

    dbutils.fs.put(output_path, json.dumps(test_data))

    print(f"  Test data created: {output_path}")
    return output_path

# COMMAND -----
# DBTITLE 1,Main Job Function
def run_json_manager_job(entities=["sales", "customers"], create_snapshots=True,
archive_old=True, days_to_keep=30):
    """
    Main job function - can be scheduled as a Databricks job
    """
    print("=" * 60)
    print("JSON FILE MANAGER JOB STARTED")
    print(f"Time: {datetime.now()}")
    print(f"Entities: {entities}")
    print("=" * 60)

    results = {
        "job_start": datetime.now().isoformat(),
        "entities_processed": [],
        "snapshots_created": [],
        "archived_count": 0,
        "errors": []
    }

```

```

}

# Step 1: Process each entity
print("\n1. PROCESSING JSON FILES")
print("-" * 40)

for entity in entities:
    print(f"\nProcessing: {entity}")
    result = process_json_files(entity)
    if result:
        results["entities_processed"].append(result)
    else:
        results["errors"].append(f"No data found for {entity}")

# Step 2: Create snapshots
if create_snapshots:
    print("\n2. CREATING SNAPSHOTS")
    print("-" * 40)

    for entity in entities:
        print(f"\nCreating snapshot for: {entity}")
        result = create_snapshot(entity)
        if result:
            results["snapshots_created"].append(result)

# Step 3: Archive old data
if archive_old:
    print("\n3. ARCHIVING OLD DATA")
    print("-" * 40)

    archived = archive_old_data(days_to_keep)
    results["archived_count"] = archived

# Step 4: Generate report
print("\n4. GENERATING REPORT")
print("-" * 40)

stats = get_file_stats()
results["file_stats"] = stats
results["job_end"] = datetime.now().isoformat()

# Save report
report_path =
f"{config.reports_path}/job_report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
dbutils.fs.put(report_path, json.dumps(results, indent=2))

print("\n" + "=" * 60)
print("JOB COMPLETED SUCCESSFULLY")
print(f"Report saved: {report_path}")
print("=" * 60)

```

```

    return results

# COMMAND -----
# DBTITLE 1,Interactive Controls (Widgets)
# Create widgets for interactive use
dbutils.widgets.text("entities", "sales,customers", "Entities to process
(comma-separated)")
dbutils.widgets.text("days_to_keep", "30", "Days to keep before archiving")
dbutils.widgets.dropdown("create_test_data", "no", ["yes", "no"], "Create test data?")
dbutils.widgets.dropdown("run_job", "yes", ["yes", "no"], "Run full job?")

# COMMAND -----
# DBTITLE 1,Interactive Execution
# Get widget values
entities = [e.strip() for e in dbutils.widgets.get("entities").split(",")]
days_to_keep = int(dbutils.widgets.get("days_to_keep"))
create_test_data_flag = dbutils.widgets.get("create_test_data") == "yes"
run_job_flag = dbutils.widgets.get("run_job") == "yes"

# Create test data if requested
if create_test_data_flag:
    print("Creating test data...")
    for entity in entities:
        create_test_data(entity, 50)
    print("Test data created successfully!")

# Run the job
if run_job_flag:
    print("Starting JSON File Manager Job...")
    results = run_json_manager_job(
        entities=entities,
        create_snapshots=True,
        archive_old=True,
        days_to_keep=days_to_keep
    )

# Display results
print("\nJOB RESULTS SUMMARY:")
print(f"Entities processed: {len(results['entities_processed'])}")
print(f"Snapshots created: {len(results['snapshots_created'])}")
print(f"Directories archived: {results['archived_count']}")

# Display file statistics
print("\nFILE STATISTICS:")
stats = results['file_stats']
for category, data in stats.items():
    if 'error' not in data:
        print(f"    {category}: {data['files']} files, {data['size_mb']} MB")
else:
    print("Job execution skipped. Set 'run_job' widget to 'yes' to execute.")

```

```

# COMMAND -----
# DBTITLE 1,Utility Functions
def list_files(directory):
    """
    List files in a directory
    """
    print(f"Files in {directory}:")
    try:
        files = dbutils.fs.ls(directory)
        for f in files[:10]: # Show first 10 files
            size_mb = f.size / (1024**2)
            print(f" {f.name} ({size_mb:.2f} MB)")
        if len(files) > 10:
            print(f" ... and {len(files) - 10} more files")
    except Exception as e:
        print(f" Error: {str(e)}")

def cleanup_temp_files():
    """
    Clean up temporary files
    """
    print("Cleaning up temporary files...")
    try:
        dbutils.fs.rm("/tmp/json_temp", recurse=True)
        print("✓ Temporary files cleaned")
    except:
        print("No temporary files to clean")

# COMMAND -----
# DBTITLE 1,Quick Actions
# Quick actions - uncomment and run specific commands as needed

# 1. List all directories
# print("Directory structure:")
# list_files(config.base_path)

# 2. Get current statistics
# print("\nCurrent file statistics:")
# stats = get_file_stats()
# print(json.dumps(stats, indent=2))

# 3. Cleanup
# cleanup_temp_files()

# 4. Manual processing of a specific entity
# result = process_json_files("sales")
# print(f"Processing result: {result}")

# 5. Manual snapshot creation
# snapshot = create_snapshot("sales")
# print(f"Snapshot result: {snapshot}")

```

```
# COMMAND -----  
# DBTITLE 1,Job Scheduling Instruction
```

Delta Lake

1. What is Delta Lake?

Delta Lake is an **open-source storage layer** that sits on top of **data lakes (ADLS, S3, GCS)** and brings **data warehouse–like reliability** to big data.

In Databricks, **Delta Lake is the default table format**.

2 Problems Delta Lake Solves

Traditional data lakes (Parquet/CSV) suffer from:

- ✗ No ACID transactions
- ✗ Data corruption during concurrent writes
- ✗ No versioning / rollback
- ✗ Complex ETL recovery

Delta Lake fixes all of these.

3 Core Features of Delta Lake (VERY IMPORTANT)

1. ACID Transactions

- Atomicity
- Consistency
- Isolation
- Durability

Multiple users/jobs can **read & write safely at the same time**

ACID Properties

1. Atomicity

Definition:

A transaction is **all-or-nothing**.

In Delta Lake

- Either **all files are committed** or **none are**
- Partial writes are **never visible**

Example

```
MERGE INTO employees t  
USING updates s  
ON t.id = s.id
```

If MERGE fails midway → **no data is written**

Implemented using **transaction commit in _delta_log**

2. Consistency

Definition:

Data moves from **one valid state to another**.

In Delta Lake

- Enforces schema rules
- Prevents invalid data writes

Example

```
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "false")
```

- ✓ Wrong data type → write fails
- ✓ Constraints maintained

3. Isolation

Definition:

Concurrent transactions do not interfere with each other.

In Delta Lake

- Uses **Optimistic Concurrency Control (OCC)**
- Readers always see a **consistent snapshot**

Example

- Job A writing data
 - Job B reading data
- ✓ Job B sees **old stable version**
 - ✓ No dirty reads

4. Durability

Definition:

Once committed, data **will not be lost**.

In Delta Lake

- Transaction logs stored in **cloud object storage**
 - Metadata safely persisted in `_delta_log`
- ✓ Even if cluster crashes → data is safe

Schema Enforcement

- Prevents bad data from being written

```
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "false")
```

- ✓ Ensures data quality

Schema Evolution

- Automatically add new columns when needed

```
df.write.format("delta") \
  .option("mergeSchema", "true") \
  .mode("append") \
  .save(path)
```

Time Travel (Versioning)

- Query older versions of data

```
SELECT * FROM delta.`/mnt/data/sales` VERSION AS OF 3;
```

- ✓ Used for **auditing, debugging, rollback**

5. Unified Batch & Streaming

- Same Delta table for **batch + streaming**

```
df.writeStream.format("delta").start(path)
```

Delta Table Internals

Delta Table Structure

```
/delta-table/
├── _delta_log/
│   ├── 000000.json
│   └── 000001.json
├── part-00001.parquet
└── part-00002.parquet
```

`_delta_log`

- Transaction log (JSON / Parquet)

- Tracks:
 - Schema
 - File additions/removals
 - Versions

✓ This enables **ACID + Time Travel**

5 Creating Delta Tables

◆ Using PySpark

```
df.write.format("delta").mode("overwrite").save("/mnt/delta/employees")
```

◆ Using SQL

```
CREATE TABLE employees  
USING DELTA  
AS SELECT * FROM source_table;
```

6 Delta Lake DML Operations

◆ INSERT

```
INSERT INTO employees VALUES (101, 'Raj', 50000);
```

◆ UPDATE

```
UPDATE employees SET salary = 60000 WHERE id = 101;
```

◆ DELETE

```
DELETE FROM employees WHERE id = 101;
```

◆ MERGE (UPSERT – MOST IMPORTANT)

```
MERGE INTO target t  
USING source s  
ON t.id = s.id  
WHEN MATCHED THEN UPDATE SET *  
WHEN NOT MATCHED THEN INSERT *;
```

✓ Widely used in **incremental pipelines**

Delta Lake Optimization Techniques

OPTIMIZE (File Compaction)

```
OPTIMIZE employees;
```

Z-ORDER (Data Skipping)

```
OPTIMIZE employees ZORDER BY (department);
```

✓ Improves query performance

VACUUM (Cleanup Old Files)

VACUUM employees RETAIN 168 HOURS;

- ⚠ Removes old data files
- ⚠ Impacts Time Travel

9. Delta Lake vs Parquet (Quick Comparison)

Feature	Parquet	Delta
ACID	✗	✓
Schema Enforcement	✗	✓
Time Travel	✗	✓
Upserts	✗	✓
Streaming + Batch	✗	✓

Delta Lake Use Cases

- Data Engineering pipelines
- CDC ingestion (Kafka → Delta)
- Data Lakehouse architecture
- Incremental loads
- Audit & compliance systems

Delta Lake in Lakehouse Architecture

Bronze (Raw) → Silver (Cleaned) → Gold (Business)
All layers use DELTA tables

Install Delta Lake

```
!pip install delta-spark
```

Initialize Spark with Delta

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("DeltaLakeDemo") \
    .config("spark.sql.extensions",
"io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog") \
```

```

        .getOrCreate()

# Create a simple DataFrame
data = [
    (1, "Alice", 25, "Engineering"),
    (2, "Bob", 30, "Sales"),
    (3, "Charlie", 35, "Marketing")
]
df = spark.createDataFrame(data, ["id", "name", "age", "department"])

# Write as Delta table
df.write.format("delta") \
    .mode("overwrite") \
    .save("/delta/employees")

# Read Delta table
delta_df = spark.read.format("delta").load("/delta/employees")
delta_df.show()

```

```

# Append new data
new_data = [(4, "Diana", 28, "HR")]
new_df = spark.createDataFrame(new_data, ["id", "name", "age", "department"])
new_df.write.format("delta").mode("append").save("/delta/employees")

# Update operation
from delta.tables import DeltaTable
delta_table = DeltaTable.forPath(spark, "/delta/employees")
delta_table.update(
    condition="department = 'Sales'",
    set={"department": "'Business Development'"})

# Delete operation
delta_table.delete("age > 40")

```

Schema Enforcement and Evolution

Schema Enforcement

Delta Lake **prevents bad data from being written** if it doesn't match the table's existing schema.

- Ensures **data quality**
- Avoids **accidental column mismatches**

How it works:

- If your Delta table has columns: id INT, name STRING, age INT
- And you try to write a DataFrame with an extra column or wrong data type, Delta will **throw an error**.

Schema Evolution

Delta Lake allows **the table schema to evolve automatically** if new columns are added in the incoming data.

- Useful for **semi-structured or changing data**
- Works only if **schema evolution is explicitly enabled**

How it works:

- You have a table: id, name, age
- Your new DataFrame has a column department
- With **schema evolution**, Delta **adds the new column** automatically

```
# Schema enforcement (will fail due to schema mismatch)
```

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
```

```
# Define schema
```

```
schema = StructType([
    StructField("id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True)
])
```

```
# Attempt to write with wrong schema will fail
```

```
wrong_data = [(5, "Eve", 50000.0)] # Salary as float instead of age as int
# This will throw an error
```

```
# Schema evolution
```

```
df_with_new_col = spark.createDataFrame(
    [(6, "Frank", 32, "IT", "M")],
    ["id", "name", "age", "department", "gender"]
)
```

```
df_with_new_col.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .save("/delta/employees")
```

Extra

```
variable010 = df.groupBy("job").agg({"salary": "avg"}).withColumnRenamed("avg(salary)", "avg_salary").show()
# Filter rows
df_filtered = df.filter(df.age > 25).show()
# Sort rows
df_sorted = df.sort(df.age.desc()).show()
# Rename columns
df_renamed = df.withColumnRenamed("salary", "income").show()
# Drop columns
df_dropped = df.drop("job").show()
# Join DataFrames
df_joined = df.join(df, ["name"], "inner").show()
# Union DataFrames
df_unioned = df.union(df).show()
# Limit rows
df_limited = df.limit(2).show()
# Drop duplicates
df_deduplicated = df.dropDuplicates(["name"]).show()
# Replace values
df_replaced = df.replace("Engineer", "Software Engineer").show()
# Cast columns
df_casted = df.withColumn("age", col("age").cast(IntegerType())).show()
# Sort rows by multiple columns
df_sorted_multiple = df.sort(df.age.desc(), df.salary.asc()).show()
# Filter rows using multiple conditions
df_filtered_multiple = df.filter((df.age > 25) & (df.salary > 60000)).show()
# Group by and aggregate
df_grouped = df.groupBy("job").agg({"salary": "avg"}).show()
# Pivot data
df_pivoted = df.groupBy("job").pivot("name").agg({"salary": "sum"}).show()
# Unpivot data
df_unpivoted = df.selectExpr("name", "age", "job", "salary", "stack(2, 'Engineer', 'Engineer', 'Manager', 'Manager') as (job, salary)").show()
# Apply a UDF
from pyspark.sql.functions import udf

def double_salary(salary):
    return salary * 2

double_salary_udf = udf(double_salary, DoubleType())
df_with_double_salary = df.withColumn("double_salary", double_salary_udf(df.salary)).show()
# Apply a lambda function
df_with_lambda = df.withColumn("age_plus_1", lambda x: x.age + 1).show()
# Apply a SQL expression
df_with_sql_expr = df.selectExpr("name", "age", "job", "salary", "salary * 2 as double_salary").show()
# Apply a Python function
def add_bonus(salary):
    return salary + 5000

df_with_bonus_python = df.withColumn("bonus", add_bonus(df.salary)).show()
# Apply a Python lambda function
```